Heart Disease Risk Prediction

End-to-End MLOps: Model Development, CI/CD, Containerisation and Kubernetes Deployment

MLOps (AIMLCZG523) - Assignment 1 | Nimishamba S | Roll No. 2025AE05517

Repository: https://github.com/Nimi060900/heart-disease-mlops

Demo video: https://drive.google.com/file/d/1tRNAtp2mYR-rczdqP4WBwBspnhAxGpmy/view?usp=drive_link

1. Overview

The project builds and operates a classifier that estimates whether a patient has heart disease from routine clinical measurements. The model is deliberately simple; the emphasis is on the engineering around it: reproducible data processing, tracked experiments, automated tests and CI, a containerised API, a Kubernetes deployment and basic monitoring.

Dataset. UCI Heart Disease, Cleveland subset: 303 patients, 13 clinical features, and a target that is binarised from 0-4 to 0 (no disease) versus 1 (disease). Collected by Detrano et al. (V.A. Medical Center Long Beach and Cleveland Clinic Foundation) and donated by D. Aha; the investigators are credited in the references, as the dataset requests.

Area	Tool	Reason
Modelling	scikit-learn Pipeline	Preprocessing and model travel together, so training and serving cannot diverge
Tracking	MLflow (SQLite backend)	Local; logs parameters, metrics, plots and models
Testing / CI	pytest, flake8, GitHub Actions	Build fails on lint or test errors; uploads logs and artifacts
Serving / packaging	FastAPI, Docker (python:3.11-slim)	Typed input validation, auto docs, isolated runtime
Orchestration	Kubernetes (Docker Desktop)	Replicas, health probes, LoadBalancer service
Monitoring	prometheus-client, Prometheus, Grafana	Request, latency and prediction metrics on a dashboard

2. Setup and Installation

Requirements: Python 3.11, Git, Docker Desktop (WSL 2 engine, Kubernetes enabled). Run from the repository root:

```
git clone https://github.com/Nimi060900/heart-disease-mlops.git && cd heart-disease-mlops
python -m venv .venv && .venv\Scripts\activate        # source .venv/bin/activate on Linux/macOS
pip install -r requirements.txt
python src/data.py          # download + clean data   (--no-download uses the committed raw file)
python -m src.train         # tune, track in MLflow, save models/model.joblib
pytest -v                   # unit tests
mlflow ui --backend-store-uri sqlite:///mlflow.db
```

3. Data Acquisition and Exploratory Analysis

`src/data.py` downloads processed.cleveland.data from UCI (skipped if present), names the columns, parses '?' as missing, binarises the target and writes data/processed/heart_clean.csv. Six values are missing, all in the discrete columns ca (4) and thal (2); they are filled with the most frequent value, since a mean would create codes that do not exist. The same imputation is repeated inside the modelling pipeline so it is fitted on training folds only. There are no duplicates. Eight columns are discrete clinical codes (treated as categorical) and five are continuous.

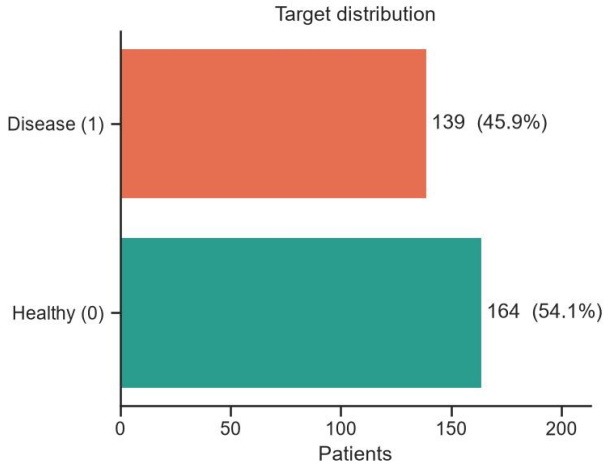


Fig. 1: Class balance (164 healthy, 139 disease)

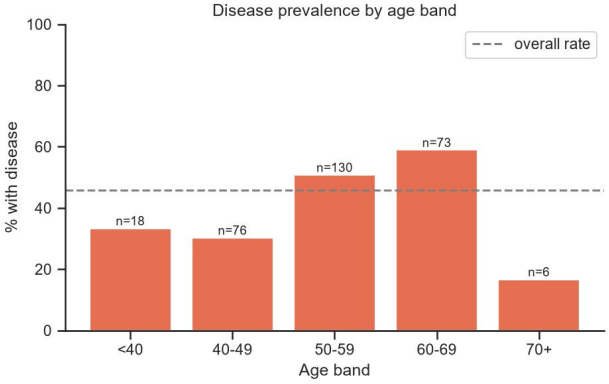


Fig. 2: Disease prevalence by age band

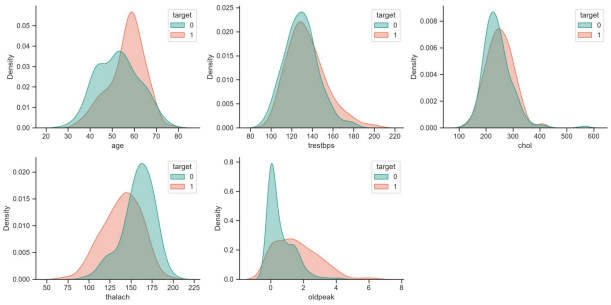


Fig. 3: Continuous features by class

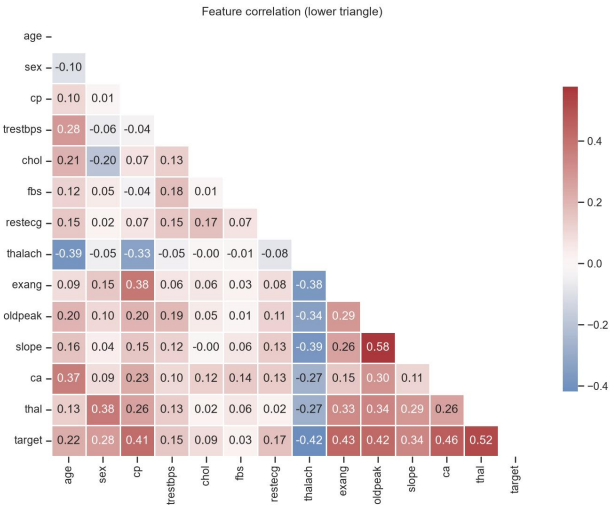


Fig. 4: Correlation matrix

Findings. The classes are close to balanced (54% / 46%), so accuracy is a meaningful metric and no resampling is needed. Patients with disease reach a lower peak heart rate (thalach) and show more ST depression (oldpeak); cholesterol and resting blood pressure overlap heavily and have a few high outliers, which motivates scaling. Prevalence is highest in the 50s and 60s (51%, 59%) and lowest in the 40s (30%); the 70+ band has only six patients and is not interpreted. thal, ca, exang, oldpeak, thalach and cp correlate most strongly with the target (|r| 0.41-0.52), and no pair of features is strongly collinear, so all 13 are kept.

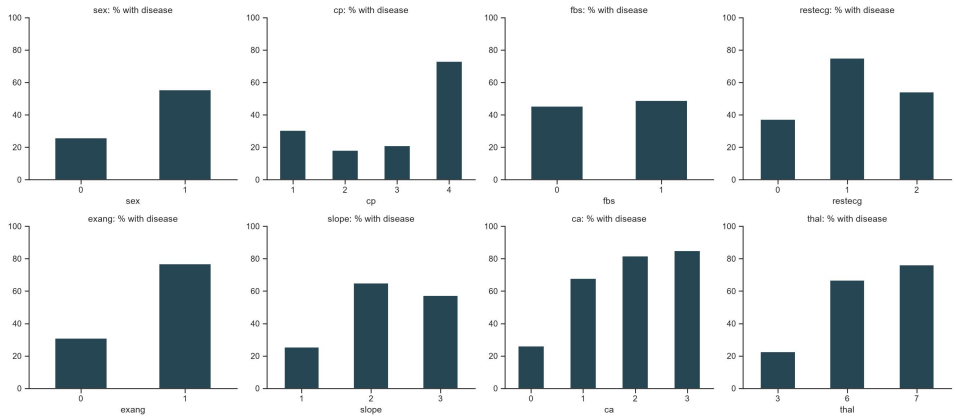


Fig. 5: Disease rate per category of the discrete features

4. Feature Engineering and Model Development

`src/features.py` builds a ColumnTransformer: numeric columns are median-imputed and standardised; categorical columns are mode-imputed and one-hot encoded with handle_unknown='ignore', so an unseen

category at serving time cannot crash the API. A stratified 80/20 split (random_state 42) holds out 61 patients; the three candidate models are tuned on the other 242 with stratified 5-fold CV and GridSearchCV scored on ROC-AUC.

Model	Search grid	Selected
Logistic Regression	C {0.01, 0.1, 1, 10}, L2	C = 1
Random Forest	n_estimators {100, 300}, max_depth {3, 5, None}, min_samples_leaf {1, 3}	100 trees, depth 5, leaf 3
Gradient Boosting	n_estimators {100, 200}, learning_rate {0.03, 0.1}, max_depth {2, 3}	100, lr 0.03, depth 2

Model	CV acc.	CV prec.	CV recall	CV AUC	Test acc.	Test recall	Test AUC
Logistic Regression	0.847	0.877	0.783	0.902	0.885	0.929	0.966
Random Forest	0.806	0.811	0.764	0.901	0.869	0.893	0.947
Gradient Boosting	0.822	0.831	0.783	0.879	0.902	0.929	0.953

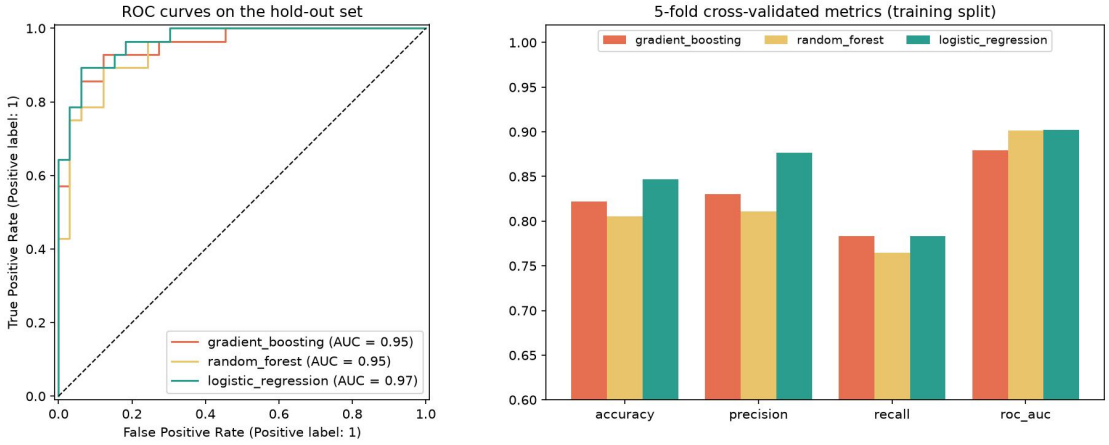
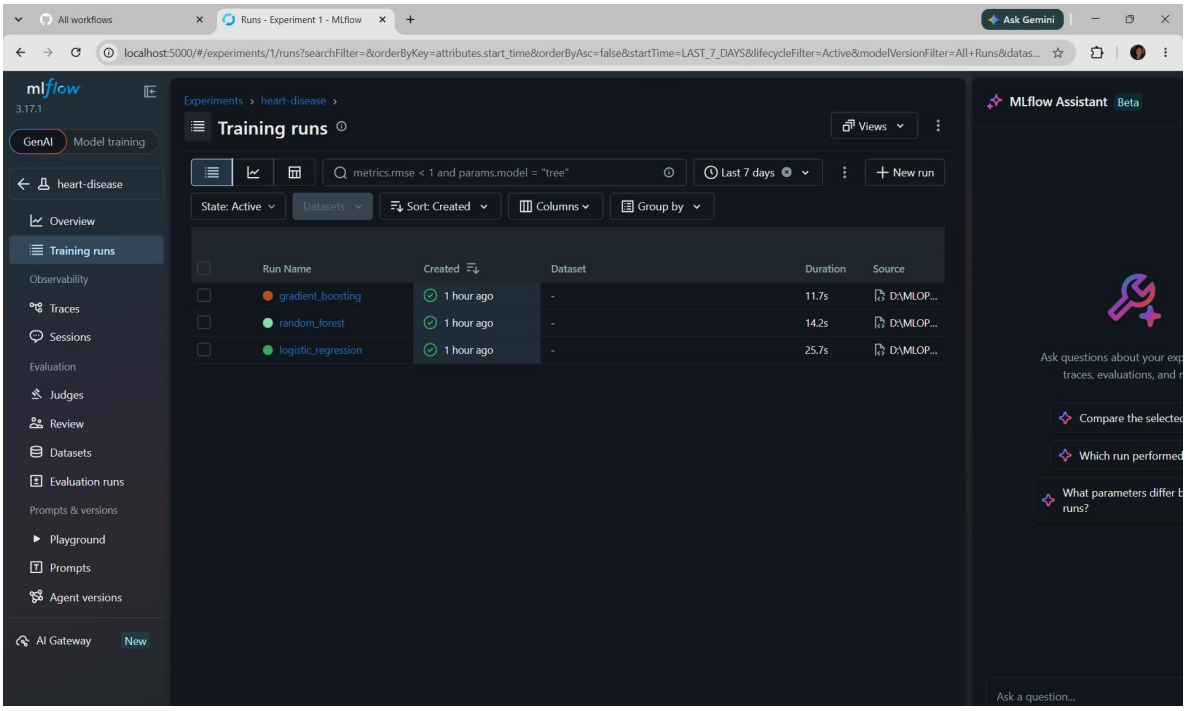


Fig. 6: ROC curves on the hold-out set (left) and cross-validated metrics (right; axis starts at 0.6)

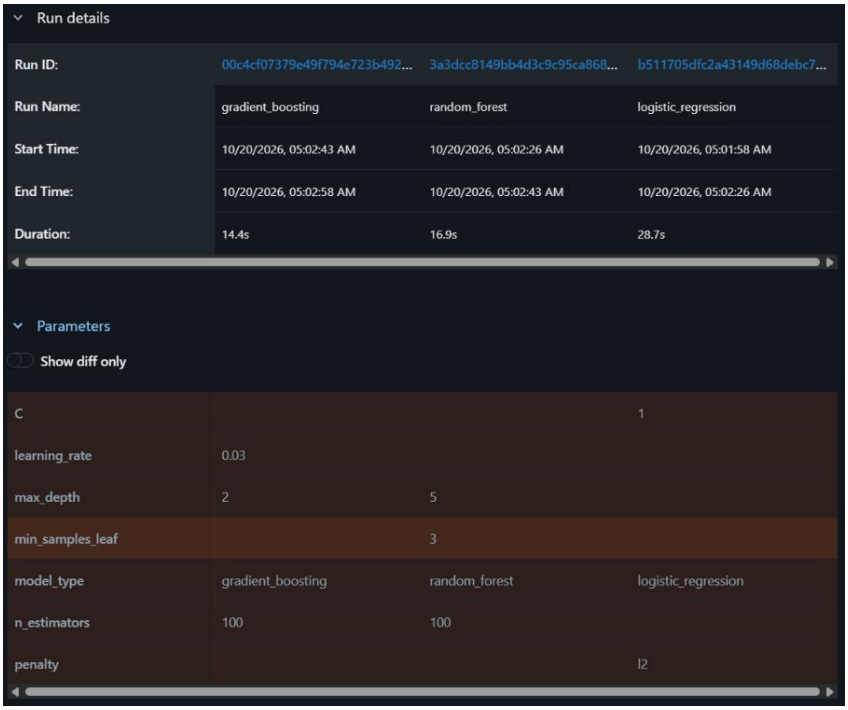
Selection. Logistic Regression was selected: its CV ROC-AUC (0.902) is tied with Random Forest (0.901), but it has the best CV accuracy and precision, ties for the best recall, has the smallest fold-to-fold variance, and is the easiest to explain. The rule is automated: rank by CV ROC-AUC (two decimals), break ties by CV recall, because missing a sick patient is the costlier error. Caveat: Gradient Boosting has the highest hold-out accuracy (0.902 vs 0.885), but with 61 test patients one patient moves accuracy by 1.6 points, so the cross-validated figures were preferred.

5. Experiment Tracking with MLflow

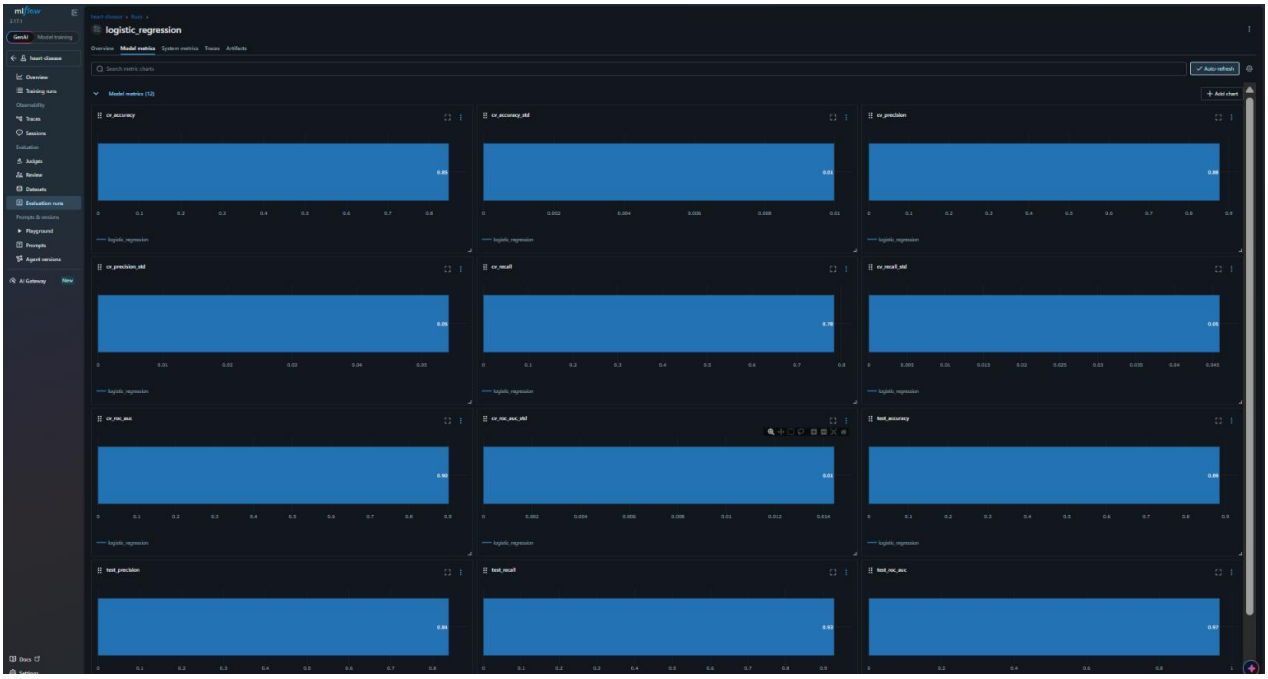
Each training run is recorded in the MLflow experiment heart-disease (SQLite backend). A run logs the tuned hyperparameters, the cross-validated accuracy, precision, recall and ROC-AUC (mean and standard deviation), the same four metrics on the hold-out set, a confusion-matrix plot, an ROC plot and the fitted pipeline. src/compare.py reads the latest run of each model back and draws Figure 6.



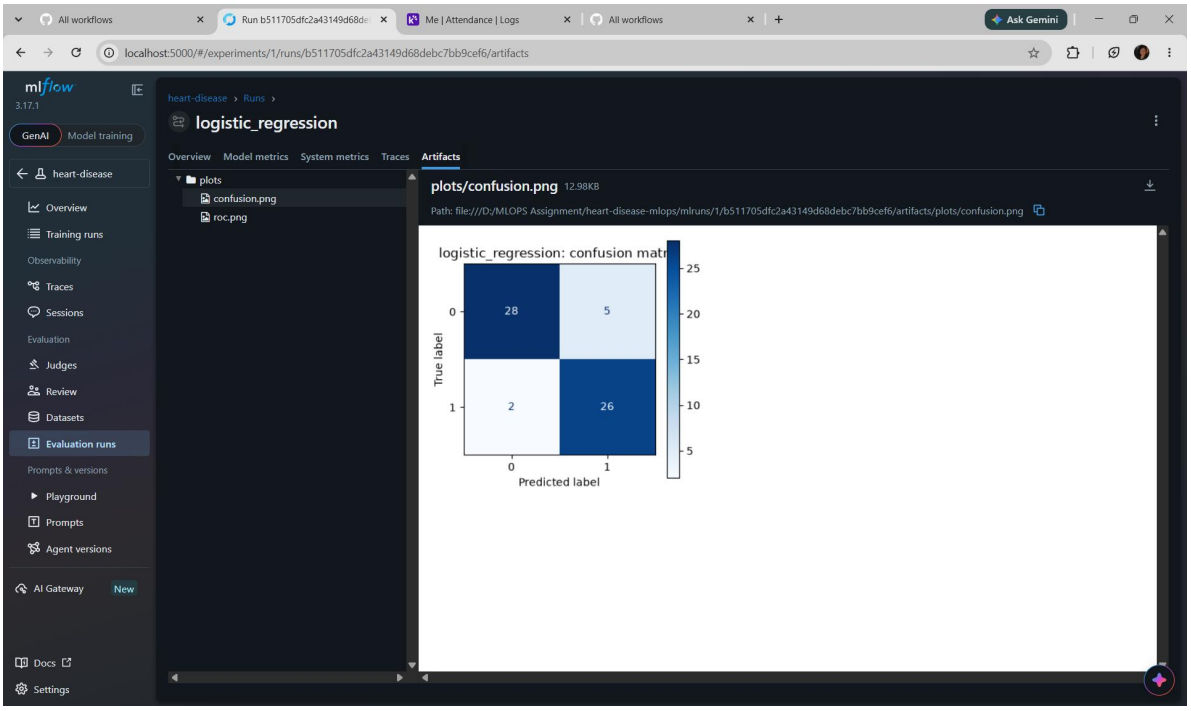
MLflow Training runs: the three final runs



Compare view: run details and chosen hyperparameters



Model metrics tab: the 12 logged metrics



Artifacts tab: logged confusion-matrix and ROC plots

6. Model Packaging and Reproducibility

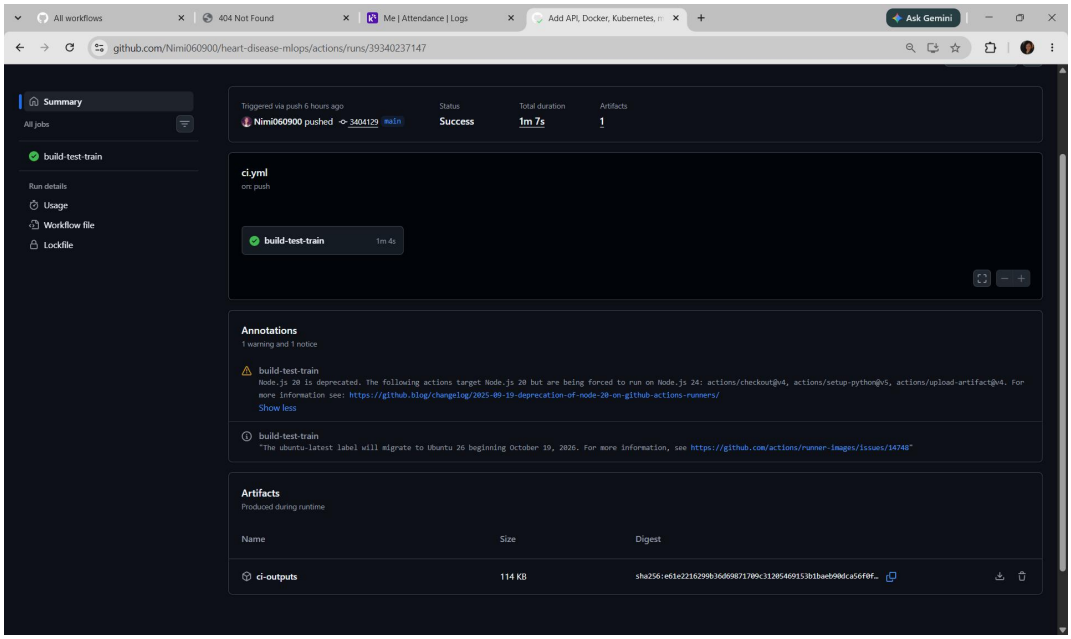
- The selected pipeline (preprocessing plus model) is saved as models/model.joblib, so the API applies exactly the training-time imputation, scaling and encoding.
- The pipeline is also logged to MLflow in cloudpickle format, because MLflow's default skops format rejected numpy.dtype; this is acceptable as the models are loaded only inside this trusted project.
- requirements.txt pins the 15 direct dependencies (requirements-api.txt holds the serving subset). All randomness uses random_state 42, so reruns reproduce the reported numbers. The cleaned dataset and download script are in the repository.

7. Automated Testing and CI/CD

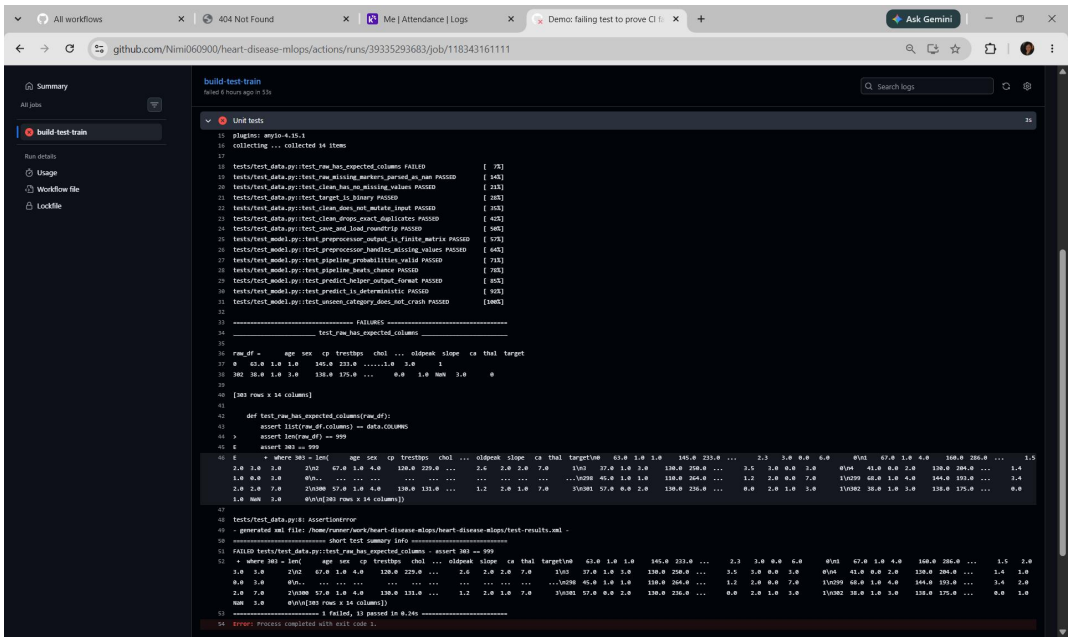
19 pytest tests: data (7: columns and row count, '?' parsed as missing, no NaNs after cleaning, binary target, input not mutated, duplicates dropped, CSV round trip), model (7: finite preprocessor output, missing-value

handling, valid probabilities, above-chance accuracy, output format, determinism, unseen category) and API (5: /health, /predict output, 422 on invalid or missing fields, /metrics).

The GitHub Actions workflow (.github/workflows/ci.yml) runs on every push to main and on pull requests, on ubuntu-latest with Python 3.11. Steps: install dependencies, flake8 lint, pytest with a JUnit report, train the models, then upload the test report, training log, model and plots as the ci-outputs artifact (if: always()). Any failing step fails the run. To demonstrate this, one assertion was deliberately broken on a separate branch and opened as a pull request (never merged); the run failed at the Unit tests step.



Successful run: Success, 1m 7s, ci-outputs artifact



Failed run: Unit tests step, 1 failed / 13 passed, exit code 1

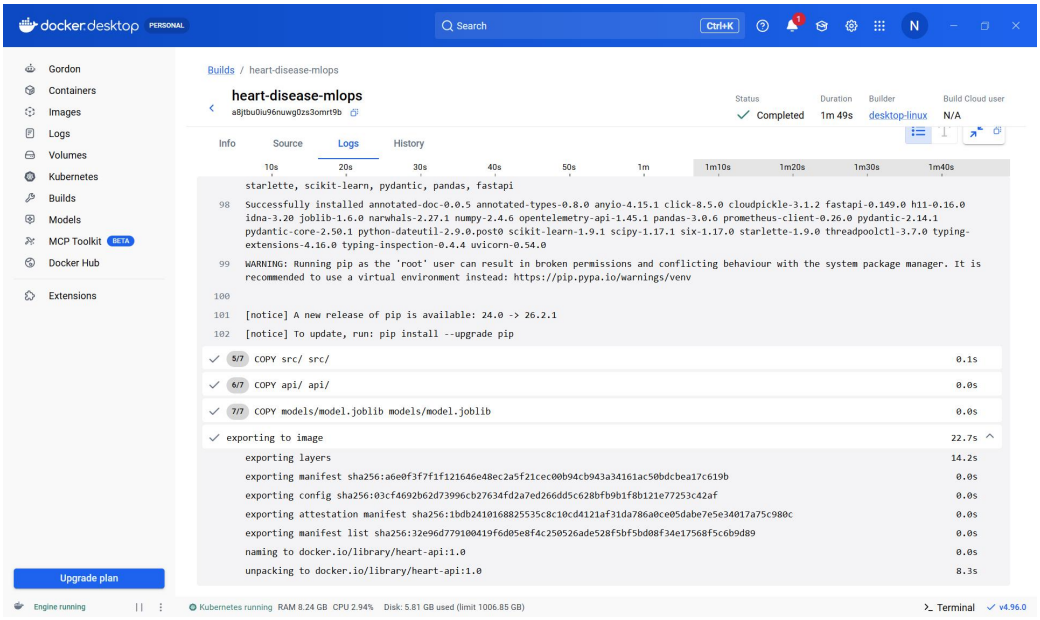
8. Model Containerisation

The FastAPI service (api/app.py) loads the pipeline once and exposes **POST /predict** (13 features as JSON, range-validated; returns prediction, confidence and probability of disease; invalid input returns 422), **GET /health** (503 if the model cannot load; used by Docker and Kubernetes probes) and **GET /metrics** (Prometheus). The Dockerfile starts from python:3.11-slim, installs only the serving dependencies, copies source and model, and defines a HEALTHCHECK; .dockerignore keeps data, notebooks and the virtual environment out of the image.

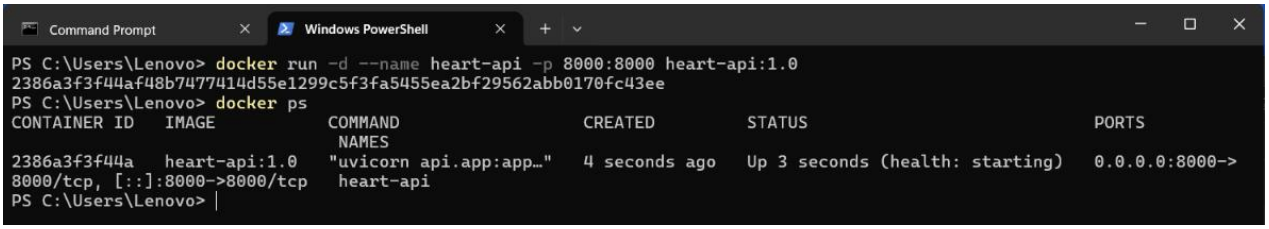
```
docker build -t heart-api:1.0 .
docker run -d --name heart-api -p 8000:8000 heart-api:1.0
```

```
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d
'{"age":63,"sex":1,"cp":1, ... "thal":6}'
# {"prediction":0,"confidence":0.8345,"probability_disease":0.1655}
```

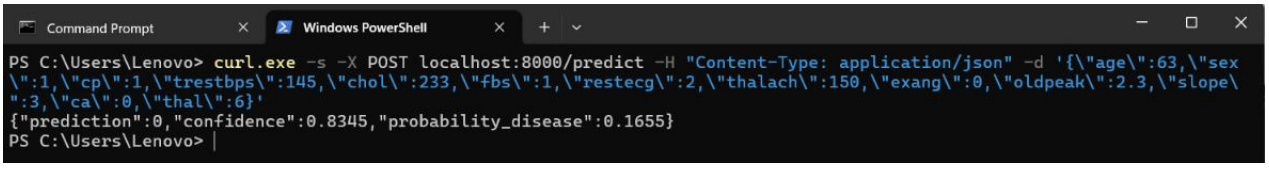
The first Cleveland patient (true label 0) is predicted healthy with 83% confidence; the second patient (true label 1) is predicted as disease with 98.6% confidence.



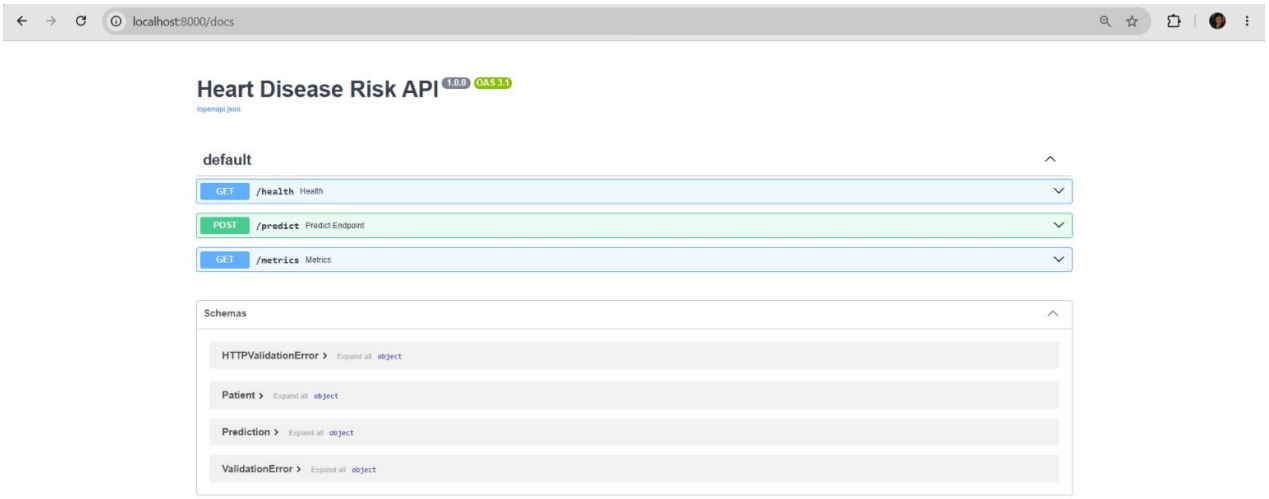
docker build log, ending with the image name heart-api:1.0



docker run and docker ps: container healthy on port 8000



/predict request and JSON response



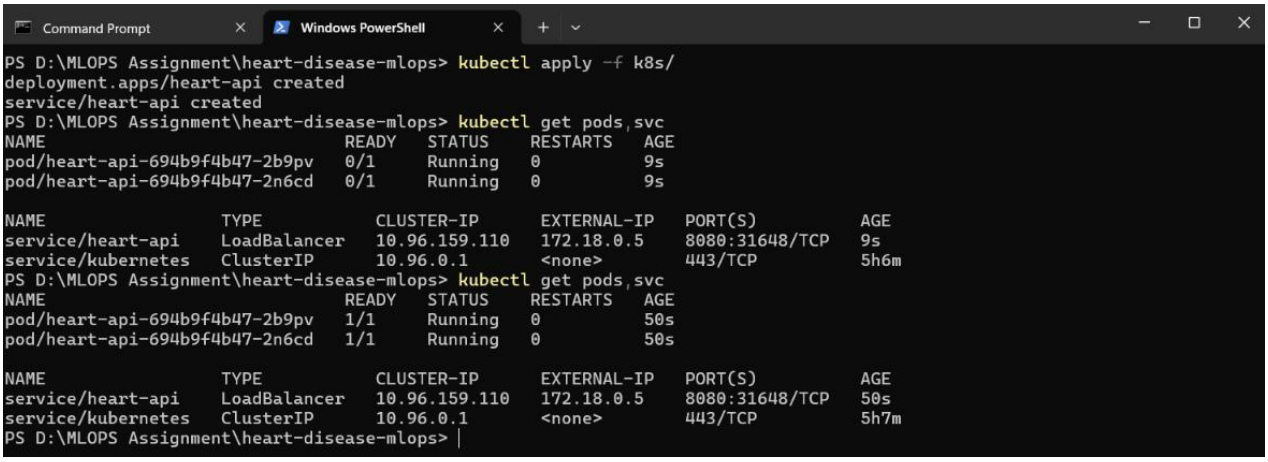
Swagger UI at /docs

9. Production Deployment on Kubernetes

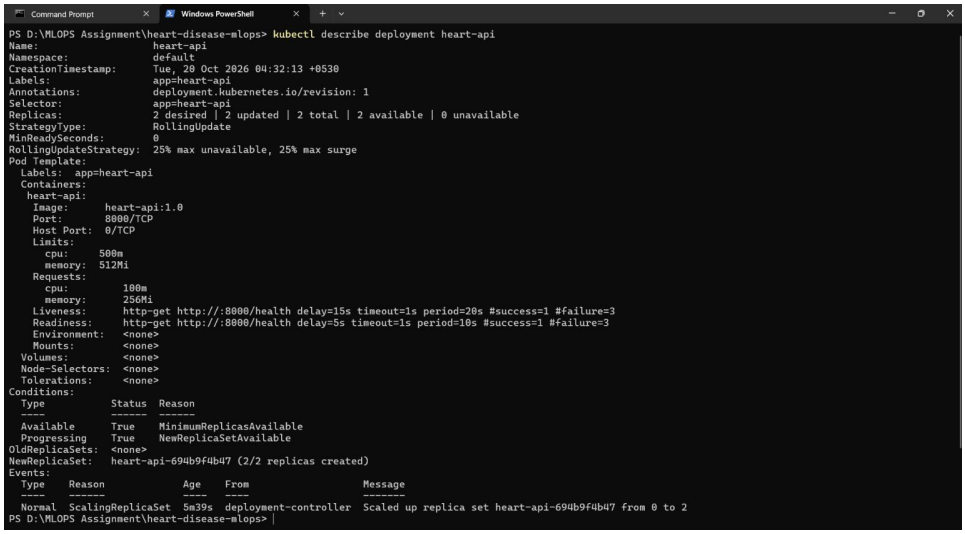
The image runs on Docker Desktop's Kubernetes cluster from two manifests in k8s/. **deployment.yaml:** 2 replicas of heart-api:1.0 (imagePullPolicy IfNotPresent, as the image is built locally), readiness and liveness

probes on /health, and CPU/memory requests (100m / 256Mi) and limits (500m / 512Mi). **service.yaml:** a LoadBalancer on port 8080 forwarding to container port 8000, published on localhost:8080. Both pods reached 1/1 Running with zero restarts, and /health and /predict answer through the load balancer.

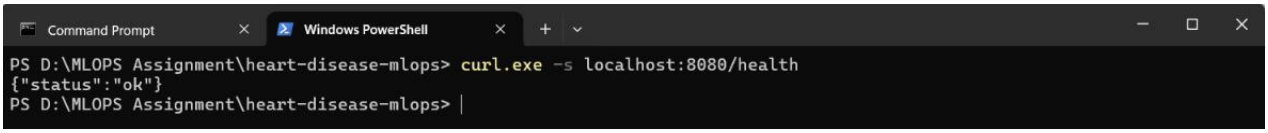
```
kubectl apply -f k8s/
kubectl get pods,svc
curl http://localhost:8080/health
```



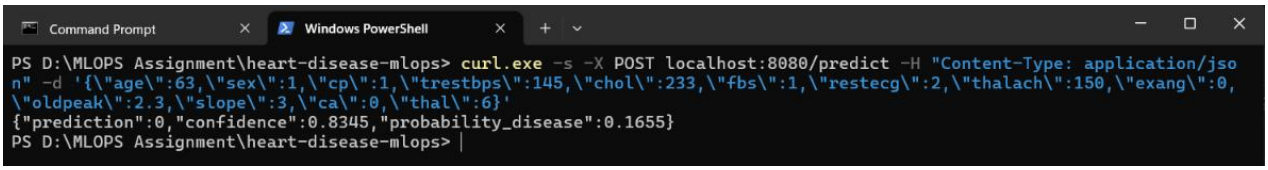
kubectl apply and get pods,svc: 2 pods Running, LoadBalancer service



kubectl describe deployment: replicas, probes, limits



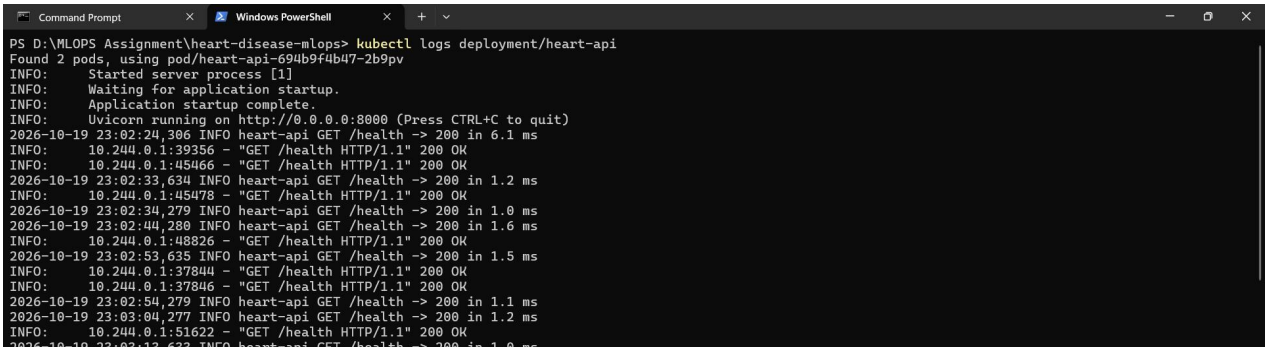
/health through the service (localhost:8080)



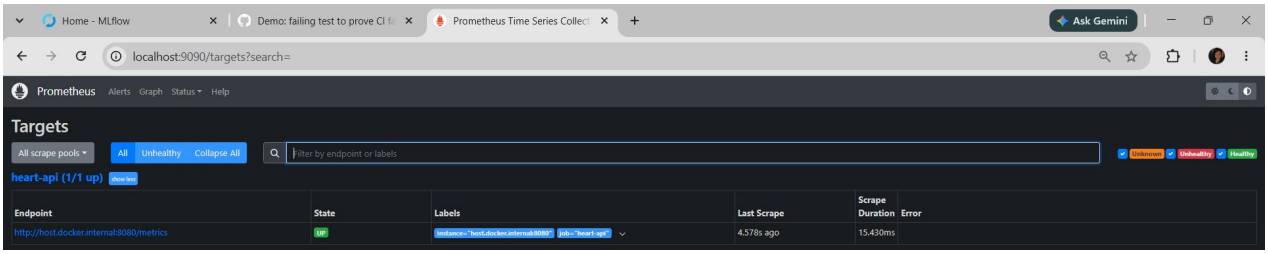
/predict through the service (localhost:8080)

10. Monitoring and Logging

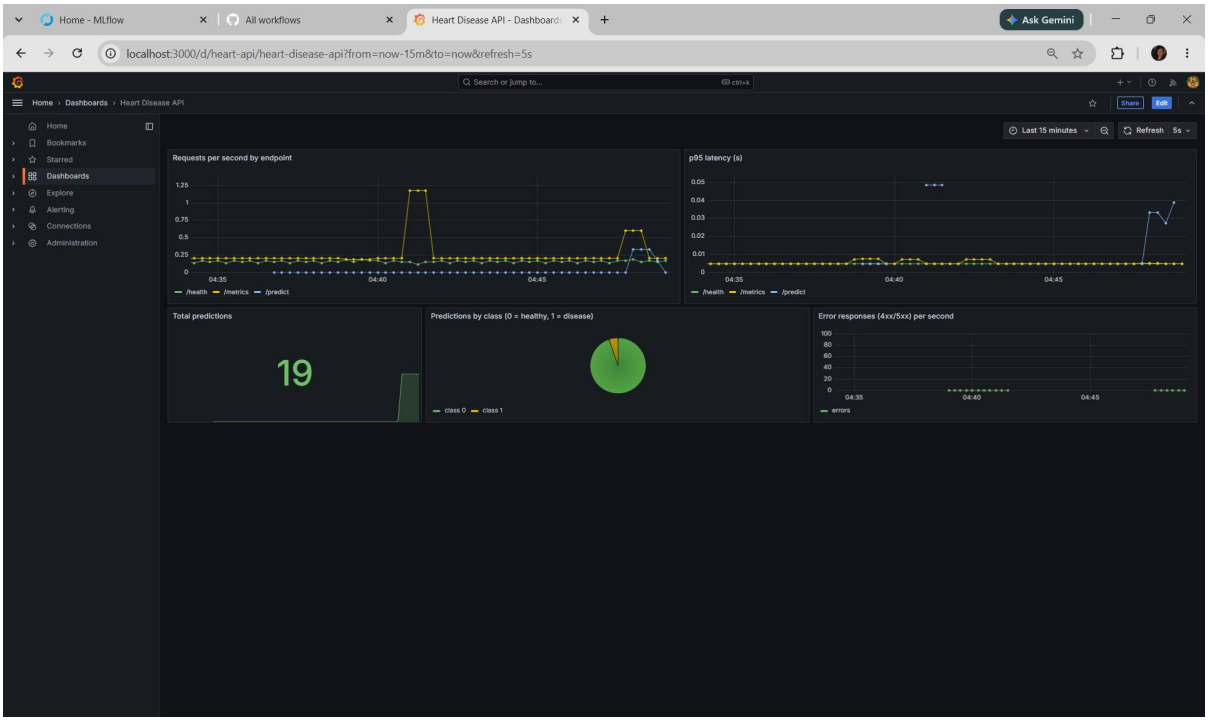
Every request is logged with timestamp, method, path, status and latency, and every prediction also logs its input and output (docker logs / kubectl logs). The API exposes api_requests_total (by endpoint and status), api_request_latency_seconds (histogram) and api_predictions_total (by predicted class). A docker-compose stack in monitoring/ runs Prometheus, which scrapes the service every 5 seconds, and Grafana with a provisioned dashboard of five panels: requests per second by endpoint, p95 latency, total predictions, predictions by class and error rate.



kubectl logs deployment/heart-api: request log lines



Prometheus targets: heart-api (1/1 up)



Grafana 'Heart Disease API' dashboard

Known limitation. Prometheus scrapes the Kubernetes service, which forwards each scrape to one of the two pods. Each pod keeps its own counters, so totals appear to jump (the dashboard shows 19 predictions although more were sent). With one replica the counters are consistent; a production setup would run Prometheus in the cluster with pod discovery.

11. Architecture

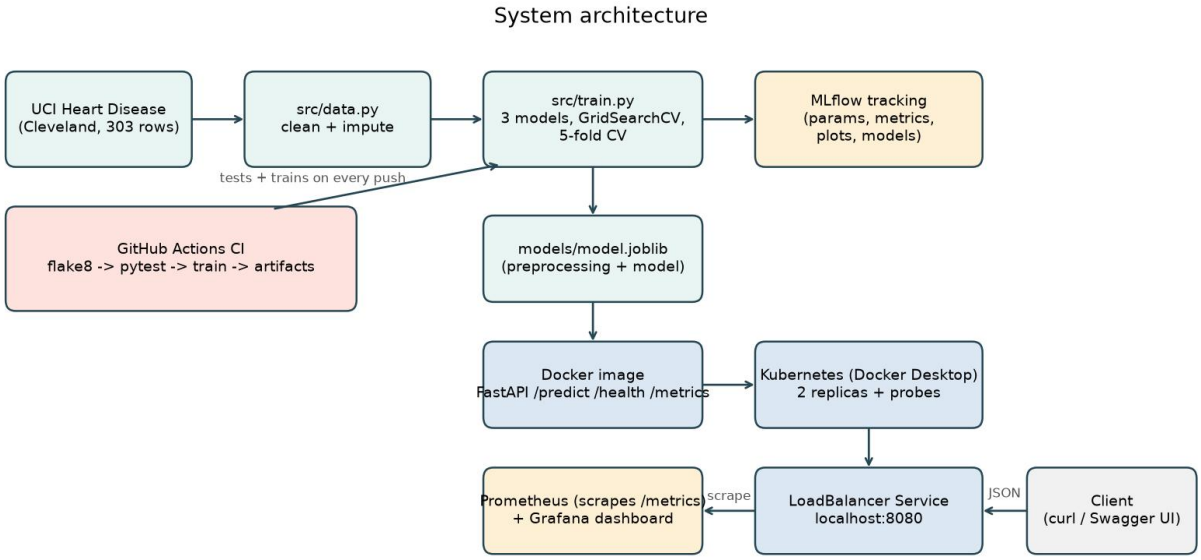


Fig. 7: System architecture from data to monitored deployment

Data is cleaned and used to train and track three models. The winning pipeline is saved as one file, packaged into a Docker image behind FastAPI and deployed to Kubernetes as two replicas behind a LoadBalancer. CI runs lint, tests and training on every push. Prometheus and Grafana observe the service through /metrics.

12. Challenges, Limitations and Future Work

- **Problems solved:** Docker Desktop needed the Windows Virtual Machine Platform feature and WSL 2; the installed MLflow blocks its old file store, so SQLite is used; MLflow's skops serialisation rejected the pipeline, so cloudpickle is used; MLflow's compare page does not show image artifacts side by side, so a separate comparison figure is generated from the stored runs.
- **Limitations:** 303 patients from one hospital is a small single-site sample; the 61-patient hold-out set is noisy, so cross-validated figures are the better guide; this is coursework, not a clinical tool; the deployment is local with no public URL.
- **Future work:** train on all four sites and test across sites; calibrate probabilities and set the threshold from a recall target; add drift monitoring, an MLflow model registry and automated image build and push in CI; deploy to a managed cluster with an Ingress and in-cluster Prometheus.

13. Conclusion

The project delivers a reproducible path from raw data to a monitored service: a tuned and tracked Logistic Regression model (CV ROC-AUC 0.90, hold-out ROC-AUC 0.97), 19 automated tests gating every commit, a container that returns predictions with confidence, and a two-replica Kubernetes deployment with request logging and a Prometheus/Grafana dashboard.

References

- Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J., Sandhu, S., Guppy, K., Lee, S. and Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. American Journal of Cardiology, 64, 304-310.
- UCI Machine Learning Repository, Heart Disease data set. Donor: David W. Aha. Principal investigators: A. Janosi (Hungarian Institute of Cardiology), W. Steinbrunn (University Hospital Zurich), M. Pfisterer (University Hospital Basel), R. Detrano (V.A. Medical Center Long Beach and Cleveland Clinic Foundation).
- Code repository: https://github.com/Nimi060900/heart-disease-mlops